

Introduction to MATLAB

Chapter 3 : Conditional statements and loops with MATLAB

Julia M. Schmidt ¹

Master Quantitative Economics
Master International Affairs and Development

September 2026

¹[julia.schmidt\[at\]dauphine.psl.eu](mailto:julia.schmidt[at]dauphine.psl.eu)

Introduction

- **Conditional statements** run a code if and only if one condition is fulfilled
- A **loop** is a programming structure that repeats a sequence of instructions until a specific condition is met

→ Conditional statements and loops are what make Matlab more than just a simple super calculator

Simple conditional statement : if...end

- Run a code if a condition is true and do nothing otherwise
- What it does:
 1. If condition TRUE, then run the code inside the loop
 2. If condition FALSE, then don't run the code inside the loop

Simple conditional statement : if...end example

What we want:

1. Generate a random number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if and only if it is bigger than 50

Simple conditional statement : if...end example

What we want:

1. Generate a random number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if and only if it is bigger than 50

```
a = randi(100,1);  
if a > 50  
    disp('a is above 50')  
end
```

More elaborate conditional statement : if...else...end

- Run a code if a condition is true and another one if false
- What it does:
 1. If condition TRUE, then run the sub-code below the if
 2. If condition FALSE, then run the sub-code after the else

Elaborate conditional statement : if...else...end example

What we want:

1. Generate a random integer number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if $a > 50$
3. Display that "a is equal or below 50" otherwise

Elaborate conditional statement : if...else...end example

What we want:

1. Generate a random integer number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if $a > 50$
3. Display that "a is equal or below 50" otherwise

```
a = randi(100,1);  
if a > 50  
    disp('a is above 50')  
else  
    disp('a is equal or below 50')  
end
```


Even more elaborate conditional statement : if...elseif...else...end

- Run a first code if a first condition is true; if false then if a second condition is true run a second code; if this second condition is false, then run a third code
- What it does:
 1. If 1st condition TRUE, then run the first sub-code below the if
 2. If condition FALSE
 - 2.1 If 2nd condition TRUE, then run the second sub-code below the elseif
 - 2.2 If 2nd condition FALSE, then run the third sub-code below the else

Even more elaborate conditional statement : example

1. Generate a random integer number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if $a > 50$
3. Display that "a is equal to 50" if $a = 50$
4. Display that "a is below 50" if $a < 50$

Even more elaborate conditional statement : example

1. Generate a random integer number between 1 and 100 drawn from an uniform distribution
2. Display that "a is above 50" if $a > 50$
3. Display that "a is equal to 50" if $a = 50$
4. Display that "a is below 50" if $a < 50$

```
a = randi(100,1);  
if a > 50  
    disp('a is above 50')  
elseif a == 50  
    disp('a is equal to 50')  
else  
    disp('a is below 50')  
end
```

Multiple conditional statements : and/or

We want now to run a code if multiple conditions are satisfied or if only one among multiple conditions is met:

- Using '&&' between two conditions will yield TRUE iff if the two conditions are TRUE
→ '&&' is equivalent to AND in logic
- Using '||' between two conditions will yield TRUE iff one of the two conditions is TRUE
→ '||' is equivalent to OR in logic

Multiple conditional statements : example

1. Generate a random integer number between 1 and 4 from an uniform distribution
2. Display that "a is odd" if... a is odd
3. Display that "a is even" if... a is even

Multiple conditional statements : example

1. Generate a random integer number between 1 and 4 from an uniform distribution
2. Display that "a is odd" if... a is odd
3. Display that "a is even" if... a is even

```
a = randi(4,1);  
if a == 1 || a == 3  
    disp('a is odd')  
else  
    disp('a is even')  
end
```

Conditional Statements : Exercise 1

1. Write a program to find the maximum between two random numbers called *x* and *y* using the *rand* function and conditional statements
2. Same question with three numbers, the last one being called *z*. Note: The probability that three numbers are equal is a zero probability event.
We now want to create a traffic light system
3. Pick up a random integer in a uniform discrete distribution taking values $\{1,2,3\}$ and store it in a variable *trafficlight*.
4. Letting 1 denotes the red light, 2 the orange and 3 the green one, write a program that shows in the console the traffic light color,e.g. "The traffic light color is red"
5. Pick up a random number in a uniform discrete distribution taking values 1,2 and store it in a variable *decision*, the latter determines the decision of the driver (2=pass the traffic light, 1=stop). Print the decision in the console.
6. Print "DANGER" mapping the decision of the driver and the color of the traffic light.

Loops for

- The for loop repeats a group of statements a **fixed** predetermined number of times
- The for loop keeps track of the number of iterations with an incrementing index variable
- Possible to nest a loop inside a loop

Loops for : example 1

1. Create a column vector of 3 random numbers uniformly distributed in $[0,1]$
2. Create a for-loop to display the content of each element of the vector

```
a = rand(3,1);  
for i = 1:length(a)  
    disp(['row ', num2str(i), ' has value ', num2str(a(i))])  
end
```

Loops for : example 2

1. Create a 3x3 matrix of random numbers uniformly distributed in [0,1]
2. Create a loop to display the content of each element of the matrix

```
a = rand(3,3);  
for i = 1:size(a,1)  
    for j = 1:size(a,2)  
        disp(['row ',num2str(i),' and column ',num2str(j),' has value ',num2str(a(i,j))]);  
    end  
end
```

Loops while

- The while loop repeats a group of statements an **indefinite** number of times as long as a condition is TRUE
 - What it does :
 - As long as a condition is TRUE, run the code inside the while-loop
 - Once the condition is FALSE, stop
- There is no index variable which is automatically incremented ($\neq$ "for" loops). It is thus needed to build and increment the index variable.

Loops while : example 1

1. Create the index variable
2. Create a column vector of 3 random numbers uniformly distributed in $[0,1]$
3. Create a while-loop to display the content of each element of the vector

Note: Increment the index variable.

Loops while : example 1

1. Create the index variable
2. Create a column vector of 3 random numbers uniformly distributed in $[0,1]$
3. Create a while-loop to display the content of each element of the vector

Note: Increment the index variable.

```
i = 0;  
a = rand(3,1);  
while i < length(a)  
    i = i+1;  
    disp(['row ',num2str(i),' has value ',num2str(a(i))]);  
end
```

Loops while : example 2

1. Create a 3x3 matrix of random numbers uniformly distributed in $[0,1]$
2. Create two index variables: one for the rows, and one for the columns.
3. Create a loop to display the content of each element of the matrix

Loops while : example 2

1. Create a 3x3 matrix of random numbers uniformly distributed in [0,1]
2. Create two index variables: one for the rows, and one for the columns.
3. Create a loop to display the content of each element of the matrix

```
a=rand(3,3); i = 0; j = 0;
while i < size(a,1)
    i = i+1;
    while j < size(a,2)
        j = j+1;
        disp(['row ',num2str(i),' and column ',num2str(j),' has value ',num2str(a(i,j))]);
    end
    j = 0; %Need to reset the value of the index variable of the nested loop
end
```

Loops: exercise 2

1. Declare $q = [3, 7, 9, 10]$; and $p = [2, 7, 8, 1]$. Create a loop which sums the elements of q

Loops: exercise 2

1. Declare $q = [3,7,9,10]$; and $p = [2,7,8,1]$. Create a loop which sums the elements of q

```
Q = 0;  
for i = 1:size(q,2)  
    Q = Q + q(i);  
end
```

Loops: exercise 2

2. Print the elements of vector `q` in reverse

Loops: exercise 2

2. Print the elements of vector `q` in reverse

```
for i = size(q,2):-1:1
    disp(['row ',num2str(i),' has value ',num2str(q(i))]);
end
```

Loops: exercise 2

3. Now, let us assume that `p` represents a vector of prices and `q` a vector of quantities. Code a for-loop which computes the total cost of the consumption basket and store it in the variable `c`.
4. Same question with a while loop
5. Pick up 50 random integers from 1 to 10 using the function `randi(.,.)`. Print elements of the vector which are above 7 and below 2
6. Pick up an integer between 1 and 100, and store it in a variable called `mystery_nb`. Create a loop where each iteration picks up a random integer between 1 and 100. As long as the latter integer is different from the mystery number, the loops keeps on running. When the loop is over, display the number of iteration.

Assignment: Getting Bitcoin Data (BCHAIN)

1. Go to Nasdaq Data Link (formerly Quandl):
<https://data.nasdaq.com/databases/BCHAIN>
2. Register for a **free API key**. You will need this to download data.
3. In MATLAB, use the `websave + readtable` approach (or the helper function) to fetch Bitcoin Market Price (code MKPRU) as a table:

```
apiKey = 'YOUR_API_KEY';  
url     = ['https://data.nasdaq.com/api/v3/datatables/QDL/BCHAIN.csv?', ...  
          'code=MKPRU&api_key=', apiKey];  
tmpcsv  = [tempname, '.csv'];  
websave(tmpcsv, url);  
d = readtable(tmpcsv);
```

Assignment Tasks

1. Plot the monthly Bitcoin Market Price. **Careful: after readtable the columns are code, date, value — there is no d.MKPRU yet.**
2. Create a loop computing the monthly return:

$$r_t = \frac{P_t}{P_{t-1}} - 1$$

Plot the series r_t .

3. Create a loop computing the 12-month (annual) return:

$$ra_t = \frac{P_t}{P_{t-12}} - 1$$

Example: 2021M1 compared to 2020M1.

4. Write two loops:
 - First loop: compute the mean of r .
 - Second loop: compute the variance of r .